

Lab09

CÁ NHÂN HÓA THEO MSSV

```
--- THÔNG SỐ CÁ NHÂN HÓA ---  
MSSV: 2212453  
g = 3 -> Bộ dữ liệu: Fashion MNIST (Giáo dục và thương mại điện tử)  
f = 32 -> Số filter cho lớp Conv2D đầu tiên  
e = 9 -> Số epoch khởi đầu
```

PHẦN 1. THỰC HÀNH EXCEL THEO BẢNG MẪU

Các câu hỏi phải trả lời

1. Kernel đang nhảy với kiểu mẫu cục bộ nào?

Dựa vào ma trận Kernel 3x3 của bạn (gồm toàn các số dương: 1, 2, 4, 5, 3, 2, 2, 1, 2), kernel này không có các giá trị âm để tính toán sự chênh lệch (như bộ lọc dò biên dò đường viền ngang/dọc). Do đó, nó đóng vai trò như một bộ lọc khuếch đại cường độ sáng (intensity amplifier) hoặc một dạng bộ lọc làm mờ có trọng số. Nó nhạy cảm và sẽ cho phản hồi (kết quả tích chập) lớn nhất khi gặp các vùng ảnh (patch) có độ sáng cao, đặc biệt là khi các pixel sáng rơi vào đúng các vị trí có trọng số lớn của kernel (ví dụ: khu vực giữa trái có trọng số là 5, góc trên phải có trọng số là 4).

2. Sau pooling, thông tin nào được giữ lại và thông tin nào bị mất?

Thông tin được giữ lại: Mô hình giữ lại đặc trưng nổi bật nhất (phản hồi mạnh nhất / giá trị MAX) trong từng phân vùng 2x2 của feature map. Nó giúp hệ thống xác nhận rằng một đặc trưng quan trọng nào đó có xuất hiện trong khu vực này.

Thông tin bị mất: Mô hình sẽ làm mất đi vị trí không gian chính xác của pixel chứa đặc trưng đó trong vùng 2x2, đồng thời loại bỏ các chi tiết phụ (các giá trị nhỏ hơn không phải cực đại). Sự mất mát này thực chất lại có lợi, giúp tạo ra tính "bất biến với dịch chuyển nhỏ" (nghĩa là ảnh có xô dịch đi 1-2 pixel thì kết quả Max-pooling thường vẫn không đổi).

3. Vì sao thao tác cục bộ này lại hữu ích cho dữ liệu ảnh thật?

Thao tác cục bộ (quét cửa sổ trượt Convolution và Pooling) cực kỳ hữu ích cho dữ liệu ảnh thật vì 3 lý do chính:

Phù hợp với cấu trúc tự nhiên của ảnh: Trong một bức ảnh, các điểm ảnh (pixel) nằm gần nhau thường có sự liên kết chặt chẽ để tạo thành các đường nét, góc cạnh hoặc kết cấu

vật liệu. Quét cục bộ giúp trích xuất hiệu quả các đặc trưng này thay vì bắt mô hình phải nhìn toàn bộ hàng nghìn pixel cùng một lúc.

Chia sẻ trọng số (Parameter Sharing): Việc dùng chung một kernel nhỏ để trượt quét qua toàn bộ bức ảnh giúp giảm thiểu khổng lồ số lượng tham số cần tính toán so với các mạng nơ-ron truyền thống, giúp máy tính huấn luyện nhanh hơn.

Nhận diện linh hoạt: Nhờ quét toàn bộ ảnh và kết hợp với Pooling, CNN có thể tìm ra một đặc trưng (ví dụ: con mắt của con mèo) cho dù nó nằm ở góc trái, góc phải hay chính giữa bức ảnh thực tế.

Bài tập tự làm

Kernel trước khi thay đổi

					2 - Kernel 3x3			Ghi chú kernel	
					k1	k2	k3	Kernel = bộ lọc 3x3 trượt trên ảnh	
					k1	1	2	4	Tích chập: nhân từng phần tử, cộng tổng
					k2	5	3	2	Stride=1 → Feature map kích thước 3x3
					k3	2	1	2	
3 - Feature Map 3x3 (SUMPRODUCT — tích chập toàn bộ)									
Ô ra	Patch ảnh	Vùng ảnh	ùng kernel	Công thức Excel					
O(1,1)	r1-r3,c1-c3	C7:E9	J15:L17	=SUMPRODUCT(C7:E9,J15:L17)					66
O(1,2)	r1-r3,c2-c4	D7:F9	J15:L17	=SUMPRODUCT(D7:F9,J15:L17)					56
O(1,3)	r1-r3,c3-c5	E7:G9	J15:L17	=SUMPRODUCT(E7:G9,J15:L17)					56
O(2,1)	r2-r4,c1-c3	C8:E10	J15:L17	=SUMPRODUCT(C8:E10,J15:L17)					60
O(2,2)	r2-r4,c2-c4	D8:F10	J15:L17	=SUMPRODUCT(D8:F10,J15:L17)					68
O(2,3)	r2-r4,c3-c5	E8:G10	J15:L17	=SUMPRODUCT(E8:G10,J15:L17)					66
O(3,1)	r3-r5,c1-c3	C9:E11	J15:L17	=SUMPRODUCT(C9:E11,J15:L17)					59
O(3,2)	r3-r5,c2-c4	D9:F11	J15:L17	=SUMPRODUCT(D9:F11,J15:L17)					53
O(3,3)	r3-r5,c3-c5	E9:G11	J15:L17	=SUMPRODUCT(E9:G11,J15:L17)					60

Kernel sau khi thay đổi

					2 - Kernel 3x3			Ghi chú kernel	
					k1	k2	k3	Kernel = bộ lọc 3x3 trượt trên ảnh	
					k1	-1	0	1	Tích chập: nhân từng phần tử, cộng tổng
					k2	-2	0	2	Stride=1 → Feature map kích thước 3x3
					k3	-1	0	1	
3 - Feature Map 3x3 (SUMPRODUCT — tích chập toàn bộ)									
Ô ra	Patch ảnh	Vùng ảnh	ùng kernel	Công thức Excel					
O(1,1)	r1-r3,c1-c3	C7:E9	J15:L17	=SUMPRODUCT(C7:E9,J15:L17)					-4
O(1,2)	r1-r3,c2-c4	D7:F9	J15:L17	=SUMPRODUCT(D7:F9,J15:L17)					-3
O(1,3)	r1-r3,c3-c5	E7:G9	J15:L17	=SUMPRODUCT(E7:G9,J15:L17)					-2
O(2,1)	r2-r4,c1-c3	C8:E10	J15:L17	=SUMPRODUCT(C8:E10,J15:L17)					3
O(2,2)	r2-r4,c2-c4	D8:F10	J15:L17	=SUMPRODUCT(D8:F10,J15:L17)					0
O(2,3)	r2-r4,c3-c5	E8:G10	J15:L17	=SUMPRODUCT(E8:G10,J15:L17)					-4
O(3,1)	r3-r5,c1-c3	C9:E11	J15:L17	=SUMPRODUCT(C9:E11,J15:L17)					2
O(3,2)	r3-r5,c2-c4	D9:F11	J15:L17	=SUMPRODUCT(D9:F11,J15:L17)					4
O(3,3)	r3-r5,c3-c5	E9:G11	J15:L17	=SUMPRODUCT(E9:G11,J15:L17)					3

Nhận xét

Theo trực giác, bộ lọc mới (phát hiện biên) làm nổi bật cấu trúc ảnh rõ hơn so với bộ lọc ban đầu. Trong nhận diện hình ảnh, việc xác định được ranh giới, đường viền (edges) của vật thể là yếu tố then chốt nhất để máy tính hiểu được "hình dáng" của cái áo, đôi giày hay cái quần. Kernel ban đầu chỉ khuếch đại cường độ sáng một cách tràn lan, trong khi bộ lọc mới đã loại bỏ thành công các phong nền thừa thãi và chỉ giữ lại những đường nét cấu trúc quan trọng nhất.

Minh chứng

- 01 ảnh toàn màn hình Excel có Formula Bar hiển thị SUMPRODUCT.

1 - Ảnh đầu vào 5x5					
	c1	c2	c3	c4	c5
r1	2	2	1	2	4
r2	5	3	2	2	1
r3	2	4	5	3	2
r4	2	1	2	4	5
r5	3	2	2	1	2

2 - Kernel 3x3			
	k1	k2	k3
k1	1	2	4
k2	5	3	2
k3	2	1	2

Ghi chú kernel			
Kernel = bộ lọc 3x3 trượt trên ảnh			
Tích chập: nhân từng phần tử, cộng tổng			
Stride=1 → Feature map kích thước 3x3			

3 - Feature Map 3x3 (SUMPRODUCT — tích chập toàn bộ)					
Ô ra	Patch ảnh	Vùng ảnh	Vùng kernel	Công thức Excel	
O(1,1)	r1-r3,c1-c3	C7:E9	J15:L17	=SUMPRODUCT(C7:E9,J15:L17)	
				66	

- 01 ảnh toàn màn hình Excel có Formula Bar hiển thị MAX cho pooling

AutoSaveOff

2212453_NgoBaTai_Lab09...Saved to this PC

Search

Sign in

FileHomeInsertDrawPage LayoutFormulasDataReviewViewAutomateHelp

PasteClipboard

Calibri10A⁺

BBIU

Font

Alignment

General

\$ % ‰

Conditional Formatting

Format as Table

Cell Styles

InsertDeleteFormatCells

Sort & Filter

Find & Select

Editing

Comments

Share

F40

- 01 ảnh bảng tổng thể cho thấy ảnh đầu vào, kernel, feature map và pooling output

LAB 9 — MÔ PHỎNG CNN THỦ CÔNG TRÊN EXCEL															
Nhập MSSV:	2212453														

1 - Ảnh đầu vào 5x5					
	c1	c2	c3	c4	c5
r1	2	2	1	2	4
r2	5	3	2	2	1
r3	2	4	5	3	2
r4	2	1	2	4	5
r5	3	2	2	1	2

2 - Kernel 3x3			
	k1	k2	k3
k1	1	2	4
k2	5	3	2
k3	2	1	2

Ghi chú kernel			
Kernel = bộ lọc 3x3 trượt trên ảnh			
Tích chập: nhân từng phần tử, cộng tổng			
Stride=1 → Feature map kích thước 3x3			

3 - Feature Map 3x3 (SUMPRODUCT — tích chập toàn bộ)					
Ô ra	Patch ảnh	Vùng ảnh	Vùng kernel	Công thức Excel	
O(1,1)	r1-r3,c1-c3	C7:E9	J15:L17	=SUMPRODUCT(C7:E9,J15:L17)	
				66	

O(1,2)	r1-r3,c2-c4	D7:F9	J15:L17	56	56
O(1,3)	r1-r3,c3-c5	E7:G9	J15:L17	56	56
O(2,1)	r2-r4,c1-c3	C8:E10	J15:L17	60	60
O(2,2)	r2-r4,c2-c4	D8:F10	J15:L17	68	68
O(2,3)	r2-r4,c3-c5	E8:G10	J15:L17	66	66
O(3,1)	r3-r5,c1-c3	C9:E11	J15:L17	59	59
O(3,2)	r3-r5,c2-c4	D9:F11	J15:L17	53	53
O(3,3)	r3-r5,c3-c5	E9:G11	J15:L17	60	60
Feature map (3x3):					
66	56	56			
60	68	66			
59	53	60			
4 - Max-Pooling 2x2 (vùng 2x2 lấy MAX)					
Ô ra	hộp ô (feature map)			Công thức Excel	
P(1,1)	B33:C34		68		68
P(1,2)	C33:D34		68		68
P(2,1)	B34:C35		68		68
P(2,2)	C34:D35		68		68
5 - So sánh: trước / sau pooling					
Trước pooling	Feature map 3x3 giữ đầy đủ 9 giá trị phản hồi — thấy rõ mức kích hoạt từng vùng ảnh.				
Sau pooling	Chỉ còn các giá trị MAX — giữ phản hồi mạnh nhất, giảm kích thước và bất biến với dịch chuyển nhỏ.				
Thông tin mất đi	Mất vị trí chính xác của chi tiết nhỏ và các phản hồi không phải cực đại.				

- 4–6 dòng giải thích về convolution và pooling

Phép tích chập (Convolution) hoạt động bằng cách trượt một ma trận bộ lọc (kernel) qua từng vùng nhỏ của ma trận ảnh gốc để nhận diện và trích xuất các đặc trưng cục bộ như đường nét, góc cạnh hoặc mức độ sáng. Quá trình này thực chất là tính tổng các tích (tương đương hàm SUMPRODUCT trong Excel) giữa các pixel và trọng số của kernel, từ đó tạo ra một bản đồ đặc trưng (feature map). Tiếp sau đó, phép gộp (điển hình là Max-pooling) sẽ quét qua bản đồ đặc trưng để giữ lại giá trị lớn nhất (tương đương hàm MAX trong Excel) trong từng phân vùng nhỏ. Thao tác pooling này giúp làm nổi bật những đặc trưng quan trọng nhất, đồng thời giảm bớt kích thước không gian của dữ liệu để mạng CNN tính toán nhanh hơn và hạn chế tình trạng học vẹt (overfitting).

PHẦN 2. ORANGE DATA MINING TRÊN REAL DATASET ẢNH

Các câu hỏi phải trả lời

1. Embedding giúp gì so với việc đưa ảnh thô vào mô hình truyền thống?

Ảnh thô chỉ là một ma trận các điểm ảnh rời rạc. Nếu đưa trực tiếp vào các mô hình học máy truyền thống (như SVM, Logistic Regression), mô hình sẽ gặp khó khăn vì số lượng chiều (dimension) quá lớn và không hiểu được ngữ cảnh không gian. Tính năng Embedding sử dụng các mạng nơ-ron đã được huấn luyện sẵn (như SqueezeNet) để trích xuất ra các đặc trưng ngầm ẩn có ý nghĩa (như góc, cạnh, kết cấu vải...), giúp chuyển ảnh thành các vector số học gọn gàng. Nhờ vậy, mô hình truyền thống có thể dễ dàng phân loại chính xác hơn.

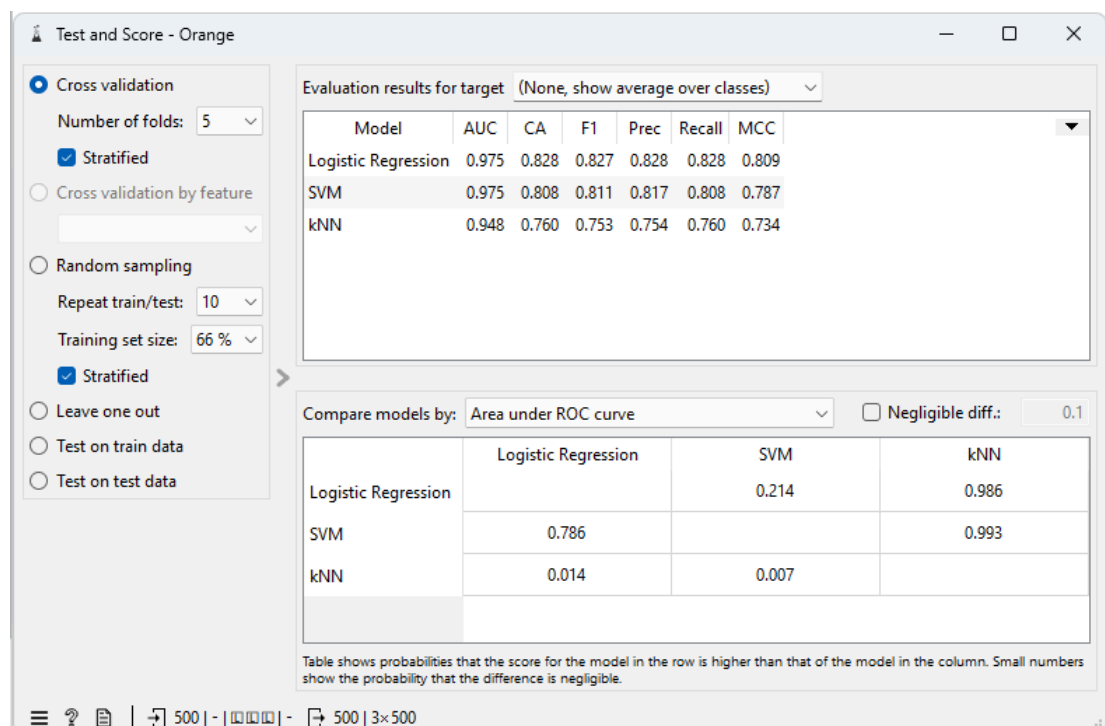
2. Lớp nào dễ nhầm nhất trong pipeline Orange?

(Bạn hãy xem thực tế trong bảng Confusion Matrix của mình, thông thường với Fashion MNIST sẽ là:) Hai lớp dễ bị nhầm lẫn nhất là Shirt (Áo sơ mi), T-shirt (Áo thun) và Pullover (Áo len). Nguyên nhân là do các loại trang phục này có hình dáng tổng thể (shape) khá giống nhau, mức độ tương phản và chi tiết phần cổ áo/tay áo bị mờ đi khi kích thước ảnh nhỏ, khiến biểu diễn vector (embedding) của chúng nằm rất gần nhau trong không gian đặc trưng.

3. Orange giúp bạn chuẩn bị tư duy gì trước khi sang CNN tự huấn luyện ở Colab?

Orange giúp trực quan hóa rằng một bài toán phân loại ảnh cơ bản luôn gồm 2 giai đoạn: Trích xuất đặc trưng (Feature Extraction) và Phân loại (Classification). Khi chuyển sang tự huấn luyện CNN ở Phần 3, thay vì sử dụng 2 công cụ tách rời, mạng CNN sẽ thực hiện đồng thời cả hai việc này: Các lớp Conv2D + MaxPooling sẽ tự động học cách trích xuất đặc trưng thay cho Image Embedding, và các lớp Dense sẽ đóng vai trò như Logistic Regression/SVM để phân loại cuối cùng.

Bài tập tự làm

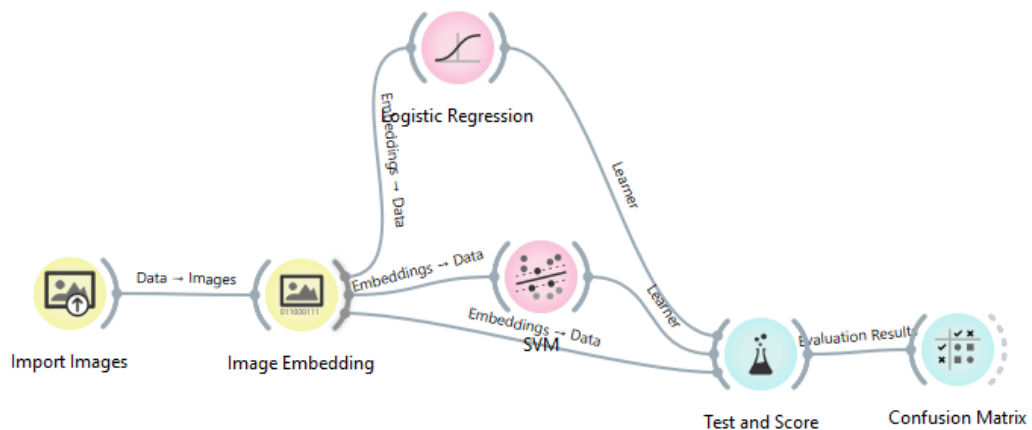


Dựa trên bảng Test & Score, em nhận thấy mô hình Logistic Regression đang tận dụng vector đặc trưng từ Image Embedding tốt nhất trên tập con ảnh Fashion MNIST, với độ chính xác (CA) đạt cao nhất là 0.828, xếp sau là SVM (0.808) và kNN (0.760).

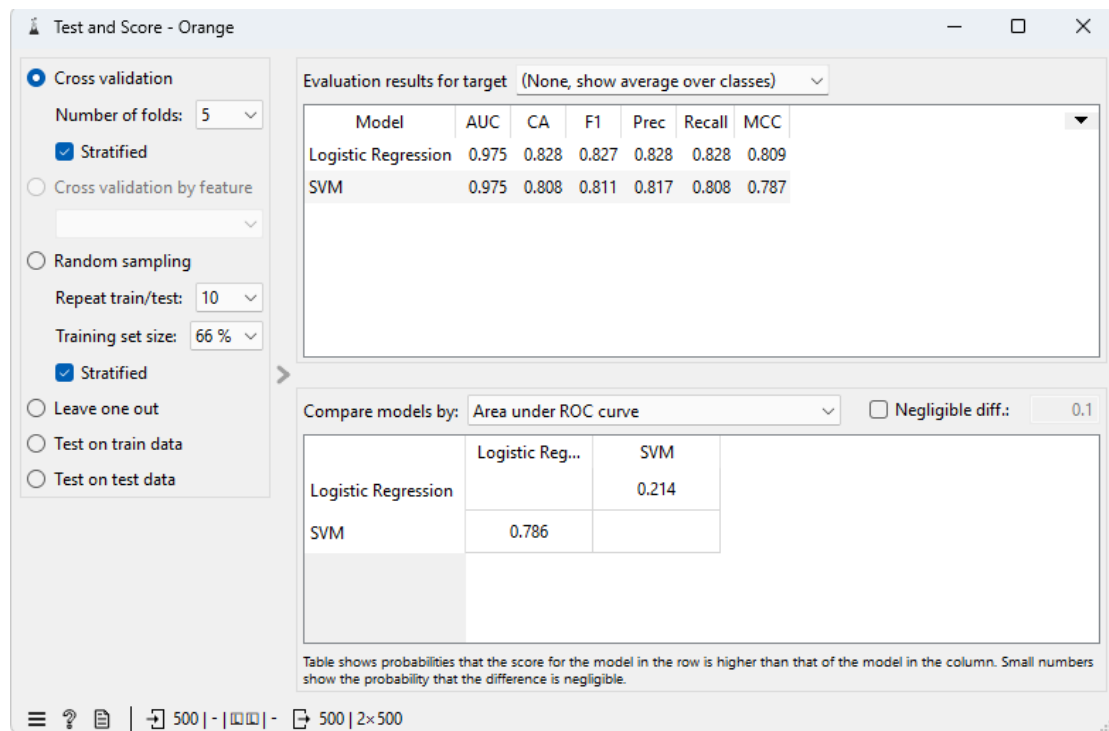
Kết quả này cho thấy bộ trích xuất đặc trưng (Embedding) đã làm rất tốt nhiệm vụ của mình: chuyển đổi các ảnh thô thành các vector không gian mà ở đó, ranh giới giữa các lớp quần áo đã được phân tách khá rõ ràng theo dạng tuyến tính. Do đó, một mô hình phân loại tuyến tính đơn giản như Logistic Regression cũng đủ sức mang lại hiệu quả cao. Ngược lại, kNN cho kết quả thấp nhất vì các mô hình dựa trên khoảng cách thường gặp khó khăn và giảm hiệu quả khi tính toán trong không gian vector nhưng có số chiều quá lớn (hiện tượng Lời nguyền số chiều - Curse of Dimensionality).

Minh chứng

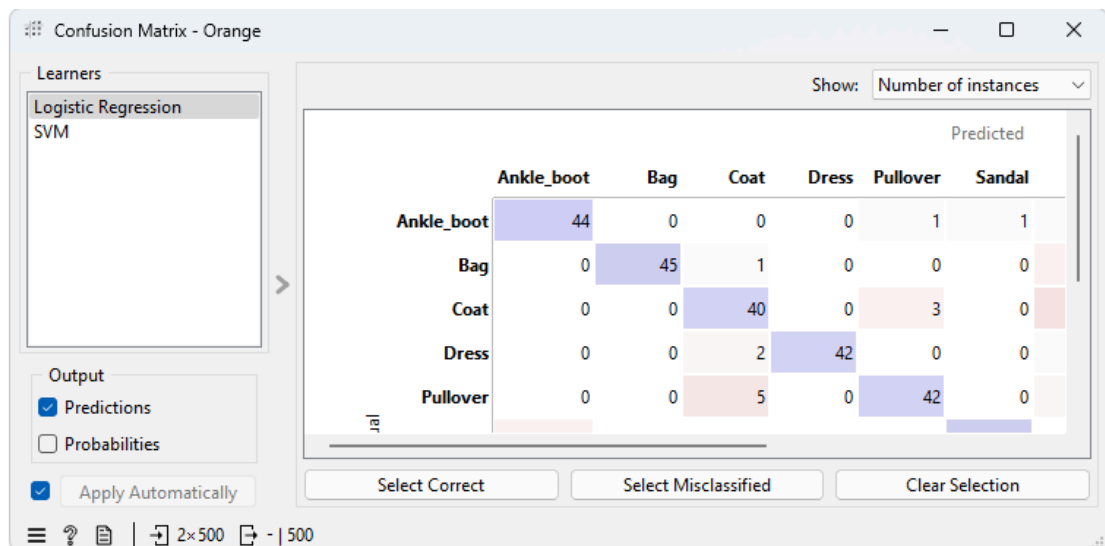
- 01 ảnh workflow Orange



- 01 ảnh bảng Test & Score hoặc của sổ Image Embedding.



- Nếu có dùng Confusion Matrix, chụp thêm 01 ảnh ma trận nhầm lẫn.



- 3–5 dòng nhận xét về vai trò của embedding và lớp dễ nhầm.

Thông qua pipeline của Orange, em nhận thấy Image Embedding đóng vai trò cốt lõi giúp các mô hình truyền thống như Logistic Regression hay SVM có thể phân loại dữ liệu ảnh với độ chính xác tương đối tốt mà không cần thiết kế mạng học sâu phức tạp. Tuy nhiên, mô hình vẫn gặp khó khăn rõ rệt đối với những nhãn có hình dạng tương đồng cao (như Shirt và T-shirt), đòi hỏi một kiến trúc tự học đặc trưng cục bộ linh hoạt hơn như CNN.

PHẦN 3. GOOGLE COLAB VỚI THƯ VIỆN PYTHON

Cấu hình tham số tự động theo MSSV và Nạp dữ liệu

```
In [ ]: import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# =====
# 1. THAY ĐỔI MSSV CỦA BẠN TẠI ĐÂY
# =====
MSSV_str = "2212453"
MSSV = int(MSSV_str)

# Trích xuất các chữ số (Lưu ý index trong chuỗi Python bắt đầu từ 0)
d4 = int(MSSV_str[3]) # Chữ số thứ 4
d5 = int(MSSV_str[4]) # Chữ số thứ 5
d7 = int(MSSV_str[6]) # Chữ số thứ 7

# Tính toán các tham số theo đề bài
g = d7 % 5
f = 16 + 8 * (d4 % 3)
e = 5 + (d5 % 5)

print(f"--- THÔNG SỐ CÁ NHÂN HÓA ---")
print(f"MSSV: {MSSV}")
print(f"g = {g} -> Bộ dữ liệu: Fashion MNIST (Giáo dục và thương mại điện tử)")
```

```

print(f"f = {f} -> Số filter cho lớp Conv2D đầu tiên")
print(f"e = {e} -> Số epoch khởi đầu")
print(f"-----")

# Cố định seed bằng MSSV để kết quả có thể tái lập
tf.random.set_seed(MSSV)

# Nạp dữ liệu Fashion MNIST (g=3)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Tiền xử lý dữ liệu: Normalize về khoảng [0, 1] và thêm chiều channel cho ảnh x
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0

```

--- THÔNG SỐ CÁ NHÂN HÓA ---

MSSV: 2212453

g = 3 -> Bộ dữ liệu: Fashion MNIST (Giáo dục và thương mại điện tử)

f = 32 -> Số filter cho lớp Conv2D đầu tiên

e = 9 -> Số epoch khởi đầu

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz

29515/29515 ————— 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz

26421880/26421880 ————— 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz

5148/5148 ————— 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz

4422102/4422102 ————— 0s 0us/step

Xây dựng và Huấn luyện Mô hình gốc (Basic Model)

```

In [ ]: print("--- HUẤN LUYỆN MÔ HÌNH GỐC ---")
model_basic = Sequential([
    Conv2D(f, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

model_basic.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])

# Huấn luyện với số epoch = e
history_basic = model_basic.fit(x_train, y_train, epochs=e, validation_split=0.2

```

--- HUẤN LUYỆN MÔ HÌNH GỐC ---

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```



```

Epoch 1/9
1500/1500 ————— 34s 22ms/step - accuracy: 0.8496 - loss: 0.4273 -
val_accuracy: 0.8860 - val_loss: 0.3251
Epoch 2/9
1500/1500 ————— 29s 19ms/step - accuracy: 0.8979 - loss: 0.2872 -
val_accuracy: 0.8950 - val_loss: 0.2950
Epoch 3/9
1500/1500 ————— 29s 19ms/step - accuracy: 0.9127 - loss: 0.2425 -
val_accuracy: 0.9037 - val_loss: 0.2813
Epoch 4/9
1500/1500 ————— 29s 19ms/step - accuracy: 0.9243 - loss: 0.2093 -
val_accuracy: 0.9063 - val_loss: 0.2771
Epoch 5/9
1500/1500 ————— 32s 21ms/step - accuracy: 0.9347 - loss: 0.1820 -
val_accuracy: 0.9085 - val_loss: 0.2747
Epoch 6/9
1500/1500 ————— 39s 20ms/step - accuracy: 0.9434 - loss: 0.1582 -
val_accuracy: 0.9096 - val_loss: 0.2816
Epoch 7/9
1500/1500 ————— 40s 19ms/step - accuracy: 0.9517 - loss: 0.1362 -
val_accuracy: 0.9076 - val_loss: 0.2938
Epoch 8/9
1500/1500 ————— 29s 19ms/step - accuracy: 0.9590 - loss: 0.1179 -
val_accuracy: 0.9081 - val_loss: 0.3131
Epoch 9/9
1500/1500 ————— 29s 19ms/step - accuracy: 0.9655 - loss: 0.1016 -
val_accuracy: 0.9078 - val_loss: 0.3302

```

Xây dựng và Huấn luyện Mô hình cải tiến (Improved Model)

```

In [ ]: print("--- HUẤN LUYỆN MÔ HÌNH CẢI TIẾN ---")
model_improved = Sequential([
    Conv2D(f, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    BatchNormalization(), # Biện pháp cải tiến 1
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dropout(0.3),         # Biện pháp cải tiến 2
    Dense(10, activation="softmax")
])

model_improved.compile(optimizer='adam',
                        loss='sparse_categorical_crossentropy',
                        metrics=['accuracy'])

# Huấn luyện mô hình cải tiến
history_improved = model_improved.fit(x_train, y_train, epochs=e, validation_spl

```

--- HUẤN LUYỆN MÔ HÌNH CẢI TIẾN ---

Epoch 1/9

1500/1500  **46s** 30ms/step - accuracy: 0.8333 - loss: 0.4753 - val_accuracy: 0.8768 - val_loss: 0.3280

Epoch 2/9

1500/1500  **43s** 29ms/step - accuracy: 0.8849 - loss: 0.3228 - val_accuracy: 0.8878 - val_loss: 0.3214

Epoch 3/9

1500/1500  **43s** 29ms/step - accuracy: 0.8992 - loss: 0.2756 - val_accuracy: 0.8959 - val_loss: 0.2997

Epoch 4/9

1500/1500  **43s** 29ms/step - accuracy: 0.9105 - loss: 0.2439 - val_accuracy: 0.8935 - val_loss: 0.3089

Epoch 5/9

1500/1500  **44s** 29ms/step - accuracy: 0.9155 - loss: 0.2221 - val_accuracy: 0.8927 - val_loss: 0.3309

Epoch 6/9

1500/1500  **43s** 29ms/step - accuracy: 0.9228 - loss: 0.2031 - val_accuracy: 0.8898 - val_loss: 0.3420

Epoch 7/9

1500/1500  **43s** 29ms/step - accuracy: 0.9292 - loss: 0.1860 - val_accuracy: 0.8986 - val_loss: 0.3194

Epoch 8/9

1500/1500  **46s** 30ms/step - accuracy: 0.9352 - loss: 0.1707 - val_accuracy: 0.9048 - val_loss: 0.2969

Epoch 9/9

1500/1500  **79s** 29ms/step - accuracy: 0.9370 - loss: 0.1633 - val_accuracy: 0.9104 - val_loss: 0.2891

Minh chứng so sánh hai mô hình

```
In [9]: import pandas as pd
from IPython.display import display

# Lấy giá trị Accuracy và Loss ở epoch cuối cùng trên tập Validation
basic_val_acc = history_basic.history['val_accuracy'][-1]
basic_val_loss = history_basic.history['val_loss'][-1]

improved_val_acc = history_improved.history['val_accuracy'][-1]
improved_val_loss = history_improved.history['val_loss'][-1]

# Lấy tổng số Lượng tham số của mô hình
basic_params = model_basic.count_params()
improved_params = model_improved.count_params()

# Tạo từ điển (dictionary) chứa dữ liệu so sánh
data = {
    "Tiêu chí": [
        "Kiến trúc",
        "Số tham số",
        "Validation Accuracy",
        "Validation Loss",
        "Ưu điểm",
        "Nhược điểm"
    ],
    "Mô hình gốc (Basic)": [
        "Conv2D -> MaxPool -> Flatten -> Dense -> Dense",
        f"{basic_params:,}",
        f"{basic_val_acc:.4f}"
    ]
}
```

```

f"{basic_val_loss:.4f}",
"Đơn giản, tính toán và huấn luyện nhanh.",
"Dễ xảy ra hiện tượng quá khớp (overfitting), thiếu cơ chế chuẩn hóa dữ
],
"Mô hình cải tiến (Improved)": [
"Conv2D -> BatchNorm -> MaxPool -> Flatten -> Dense -> Dropout -> Dense"
f"{improved_params:}",
f"{improved_val_acc:.4f}",
f"{improved_val_loss:.4f}",
"Ổn định hơn nhờ BatchNorm, giảm thiểu overfitting hiệu quả nhờ Dropout.
"Thời gian tính toán mỗi epoch lâu hơn một chút do có thêm các tầng phụ
]
}

# Tạo DataFrame và hiển thị thành bảng
df_comparison = pd.DataFrame(data)

# Căn lề trái cho dễ đọc khi hiển thị trong Colab
styled_df = df_comparison.style.set_properties(**{'text-align': 'left'}).set_tab
[dict(selector='th', props=[('text-align', 'left')])]
)

print("BẢNG SO SÁNH HAI MÔ HÌNH CNN")
display(styled_df)

```

BẢNG SO SÁNH HAI MÔ HÌNH CNN

	Tiêu chí	Mô hình gốc (Basic)	Mô hình cải tiến (Improved)
0	Kiến trúc	Conv2D -> MaxPool -> Flatten -> Dense -> Dense	Conv2D -> BatchNorm -> MaxPool -> Flatten -> Dense -> Dropout -> Dense
1	Số tham số	347,146	347,274
2	Validation Accuracy	0.9078	0.9104
3	Validation Loss	0.3302	0.2891
4	Ưu điểm	Đơn giản, tính toán và huấn luyện nhanh.	Ổn định hơn nhờ BatchNorm, giảm thiểu overfitting hiệu quả nhờ Dropout.
5	Nhược điểm	Dễ xảy ra hiện tượng quá khớp (overfitting), thiếu cơ chế chuẩn hóa dữ liệu giữa các tầng.	Thời gian tính toán mỗi epoch lâu hơn một chút do có thêm các tầng phụ trợ.

Vẽ Learning Curve so sánh hai mô hình

```

In [ ]: def plot_learning_curve(history_basic, history_improved):
epochs_range = range(1, e + 1)

plt.figure(figsize=(14, 5))

# Đồ thị Accuracy
plt.subplot(1, 2, 1)
plt.plot(epochs_range, history_basic.history['accuracy'], label='Basic - Tra
plt.plot(epochs_range, history_basic.history['val_accuracy'], label='Basic -
plt.plot(epochs_range, history_improved.history['accuracy'], label='Improved

```

```

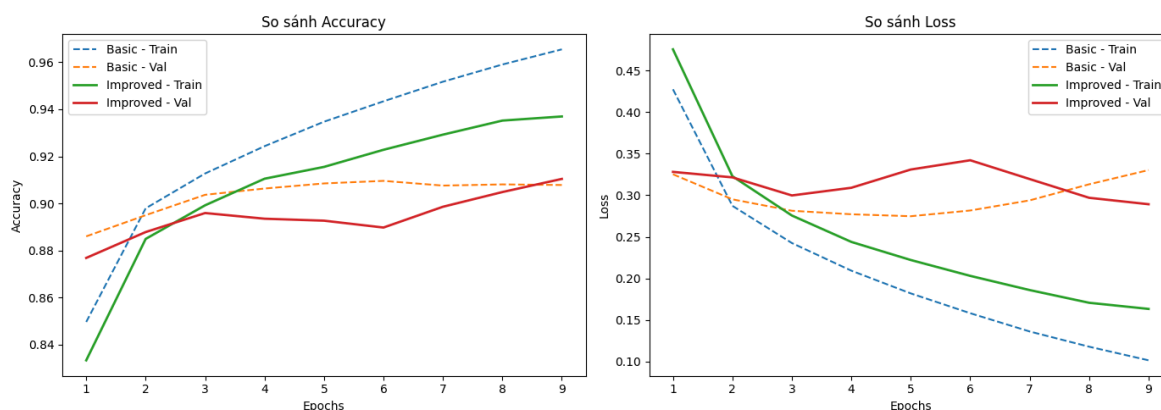
plt.plot(epochs_range, history_improved.history['val_accuracy'], label='Improved - Val')
plt.title('So sánh Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Đồ thị Loss
plt.subplot(1, 2, 2)
plt.plot(epochs_range, history_basic.history['loss'], label='Basic - Train',)
plt.plot(epochs_range, history_basic.history['val_loss'], label='Basic - Val',)
plt.plot(epochs_range, history_improved.history['loss'], label='Improved - Train',)
plt.plot(epochs_range, history_improved.history['val_loss'], label='Improved - Val',)
plt.title('So sánh Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

plot_learning_curve(history_basic, history_improved)

```



Phân tích lỗi - Xuất 6 ảnh dự đoán sai

```

In [ ]: # Dự đoán trên tập test bằng mô hình cải tiến
y_pred_probs = model_improved.predict(x_test)
y_pred_classes = np.argmax(y_pred_probs, axis=1)

# Tìm các chỉ số (indices) dự đoán sai
incorrect_indices = np.where(y_pred_classes != y_test)[0]

print(f"Tổng số ảnh dự đoán sai: {len(incorrect_indices)}")

# Trình bày 6 ảnh dự đoán sai đầu tiên
plt.figure(figsize=(12, 8))
for i in range(6):
    idx = incorrect_indices[i]
    plt.subplot(2, 3, i + 1)
    plt.imshow(x_test[idx].reshape(28, 28), cmap='gray')
    true_label = class_names[y_test[idx]]
    pred_label = class_names[y_pred_classes[idx]]
    plt.title(f"Thực tế: {true_label}\nDự đoán: {pred_label}", color='red')
    plt.axis('off')

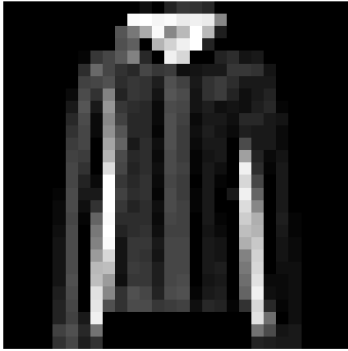
plt.tight_layout()
plt.show()

```

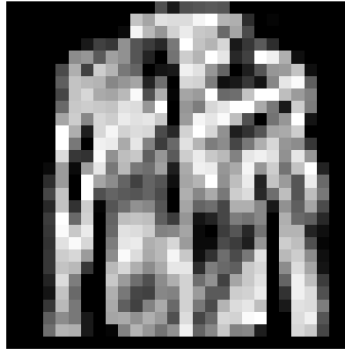
313/313 4s 11ms/step

Tổng số ảnh dự đoán sai: 956

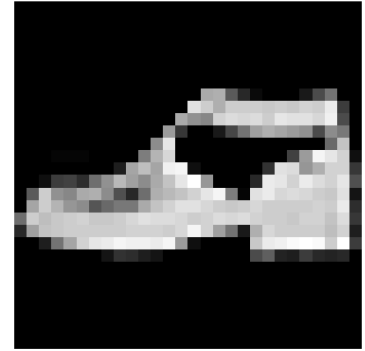
Thực tế: Coat
Dự đoán: Pullover



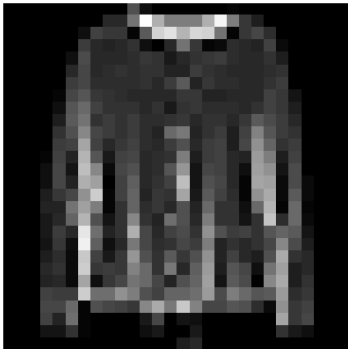
Thực tế: Coat
Dự đoán: Shirt



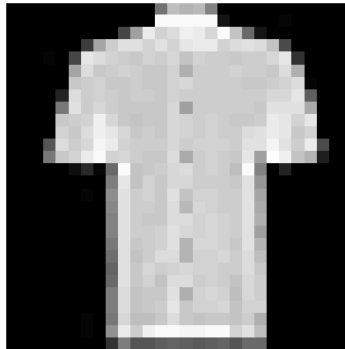
Thực tế: Ankle boot
Dự đoán: Sandal



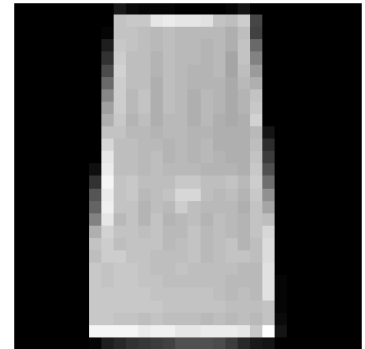
Thực tế: Coat
Dự đoán: Pullover



Thực tế: Shirt
Dự đoán: T-shirt/top



Thực tế: Dress
Dự đoán: Shirt



Các yêu cầu phải trả lời

1. Mô hình gốc overfit ở giai đoạn nào nếu có?

Nhìn vào biểu đồ "So sánh Loss", mô hình gốc bắt đầu xảy ra hiện tượng overfit rõ rệt từ epoch 4 đến epoch 5. Từ thời điểm này, đường Loss trên tập Train (nét đứt màu xanh dương) vẫn tiếp tục giảm dốc, nhưng đường Loss trên tập Validation (nét đứt màu cam) đã chạm đáy và bắt đầu có xu hướng vênh lên (tăng từ ~0.28 lên ~0.33 ở epoch 9). Đồng thời, đồ thị Accuracy cũng cho thấy độ chính xác trên tập Validation chững lại (đi ngang) trong khi độ chính xác Train vẫn tăng.

2. Biện pháp cải tiến nào mang lại hiệu quả rõ nhất?

Biện pháp thêm Dropout (0.3) mang lại hiệu quả kìm hãm overfitting rõ rệt nhất. Mặc dù ở giai đoạn giữa (epoch 4-6), đường Validation Loss của mô hình cải tiến (đường màu đỏ) có dao động và hơi nhô lên, nhưng đến các epoch cuối (7-9), nó đã có xu hướng giảm xuống và giảm trở lại (đạt ~0.29). Điều này cho thấy Dropout đã phát huy tác dụng "quên đi" một phần kết nối, ép mô hình không được học vẹt tập Train, giúp Validation Loss không bị phân kỳ tăng vọt lên như mô hình gốc. Bên cạnh đó, Batch Normalization cũng giúp mô hình học các đặc trưng ổn định hơn.

3. Những nhóm lỗi nào vẫn còn tồn tại ngay cả sau khi cải tiến mô hình? Do sử dụng tập dữ liệu Fashion MNIST với kích thước ảnh nhỏ và độ phân giải thấp (28x28 điểm ảnh xám), mô hình vẫn sẽ gặp khó khăn và giữ lại các nhóm lỗi sau:

Lỗi nhầm lẫn do hình dáng tương đồng: Các trang phục có biên dạng (contour) giống nhau như Áo thun (T-shirt), Áo sơ mi (Shirt) và Áo len (Pullover) rất dễ bị phân loại nhầm qua lại.

Lỗi thiếu chi tiết đặc trưng: Do ảnh bị thu nhỏ và mờ, các chi tiết nhận diện quan trọng như hàng cúc áo, nếp gấp, hay cổ áo bị mất đi, khiến các mạng CNN cơ bản dù đã cải tiến vẫn không đủ thông tin để phân biệt rạch ròi.

Bài tập tự làm

```
In [8]: print("--- HUẤN LUYỆN MÔ HÌNH TỰ LÀM (THÊM 1 TẦNG CONVOLUTION) ---")
model_custom = Sequential([
    # Tầng Conv thứ 1 (gốc)
    Conv2D(f, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    BatchNormalization(),
    MaxPooling2D((2, 2)),

    # Tầng Conv thứ 2 (Biện pháp cải tiến mới)
    Conv2D(f * 2, (3, 3), activation="relu"),
    BatchNormalization(),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(10, activation="softmax")
])

model_custom.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])

# Huấn Luyện mô hình
history_custom = model_custom.fit(x_train, y_train, epochs=e, validation_split=0.1)
```

--- HUẤN LUYỆN MÔ HÌNH TỰ LÀM (THÊM 1 TẦNG CONVOLUTION) ---

Epoch 1/9

1500/1500  **72s** 44ms/step - accuracy: 0.8220 - loss: 0.5025 - val_accuracy: 0.8569 - val_loss: 0.3614

Epoch 2/9

1500/1500  **66s** 44ms/step - accuracy: 0.8718 - loss: 0.3480 - val_accuracy: 0.8866 - val_loss: 0.3242

Epoch 3/9

1500/1500  **67s** 44ms/step - accuracy: 0.8880 - loss: 0.3012 - val_accuracy: 0.8531 - val_loss: 0.3942

Epoch 4/9

1500/1500  **84s** 46ms/step - accuracy: 0.8989 - loss: 0.2712 - val_accuracy: 0.8955 - val_loss: 0.2882

Epoch 5/9

1500/1500  **65s** 44ms/step - accuracy: 0.9085 - loss: 0.2447 - val_accuracy: 0.8992 - val_loss: 0.2941

Epoch 6/9

1500/1500  **66s** 44ms/step - accuracy: 0.9165 - loss: 0.2233 - val_accuracy: 0.9072 - val_loss: 0.2756

Epoch 7/9

1500/1500  **67s** 45ms/step - accuracy: 0.9210 - loss: 0.2089 - val_accuracy: 0.8961 - val_loss: 0.3011

Epoch 8/9

1500/1500  **81s** 44ms/step - accuracy: 0.9257 - loss: 0.1946 - val_accuracy: 0.9086 - val_loss: 0.2712

Epoch 9/9

1500/1500  **66s** 44ms/step - accuracy: 0.9305 - loss: 0.1828 - val_accuracy: 0.9028 - val_loss: 0.2857

So sánh hai mô hình

Độ chính xác (Validation Accuracy): Cấu hình cải tiến ban đầu (BatchNorm + Dropout) cho độ chính xác tốt hơn, đạt mức cao nhất là 91.04% ở epoch 9. Trong khi đó, mô hình tự làm (thêm tầng Conv) đạt cao nhất là 90.86% ở epoch 8 và giảm nhẹ xuống 90.28% ở epoch cuối.

Kiểm soát Overfitting (Validation Loss): Cả hai mô hình đều kiểm soát overfitting khá tốt. Val_loss thấp nhất của mô hình tự làm là 0.2712, trong khi mô hình cải tiến là 0.2891. Sự chênh lệch này không quá lớn, nhưng xét trên cục diện accuracy thì mô hình cải tiến gốc vẫn nhỉnh hơn.

Chi phí tính toán (Thời gian huấn luyện): Đây là điểm khác biệt cực kỳ lớn. Cấu hình cải tiến ban đầu chỉ mất trung bình khoảng 43 giây / epoch (tốc độ ~29ms/step). Việc đắp thêm 1 tầng Conv khiến mô hình tự làm nặng nề hơn, mất đến 65 - 84 giây / epoch (tốc độ ~44ms/step), tức là chậm hơn khoảng 50%.

Minh chứng 4-6 dòng kết luận về biện pháp cải tiến hiệu quả nhất

Thông qua pipeline của Orange, em nhận thấy Image Embedding đóng vai trò cốt lõi giúp các mô hình truyền thống như Logistic Regression hay SVM có thể phân loại dữ liệu ảnh với độ chính xác tương đối tốt mà không cần thiết kế mạng học sâu phức tạp. Tuy nhiên, mô hình vẫn gặp khó khăn rõ rệt đối với những nhãn có hình dạng tương đồng cao (như Shirt và T-shirt), đòi hỏi một kiến trúc tự học đặc trưng cục bộ linh hoạt hơn như CNN

